

AI RACE

Comprehensive Game Design & Product Vision Claude Code Brief

Purpose: Describe the intended game experience, systems, features, tone, and design philosophy while leaving technical implementation decisions to the development AI.

1. Core Concept

AI Race is an open-ended strategy/simulation game where each player controls an entire artificial intelligence company. The player is not a CEO character or individual employee; the player effectively is the company, similar to how a grand-strategy game treats the player as an entire country.

The game combines AI development, AI competition, and company competition. Players create, develop, release, and improve their own language model while growing the company around it.

2. Central Design Principle

The game must not feel like a spreadsheet where **spend more money** → **number goes up** → **AI gets better**.

Instead, player development decisions should shape the resulting model. The player should be able to interact with that model, observe its strengths and weaknesses, and decide what to improve next.

The desired loop: player decisions → model development → distinct AI behavior → interaction → evaluation → improvement.

The player is genuinely developing a miniature language model. The game does not simulate AI development through a predetermined progression of upgrades or a fixed set of model states. Player decisions influence the actual development and training of the elementary language model.

Because AI Race is fundamentally a sandbox, the player's AI does not have a predetermined final form. There is no fixed development path that says what the AI can ultimately become, and there should not be an arbitrary "maximum level" at which development simply stops.

The model begins extremely small and limited, but continued training, experimentation, different data mixtures, development decisions, and successive iterations can continue changing and improving it. The game therefore represents an ongoing process of AI development rather than a finite technology tree.

The player may eventually reach practical limitations imposed by resources, computing capacity, data, training time, or the inherent limitations of the miniature model, but these should emerge from the underlying system rather than from an artificial game rule such as "Level 20 is the maximum."

The important distinction is:

The game has limits because the simulated AI and the player's resources have limits — not because the game has a predefined list of upgrades that determines exactly how far the player is allowed to develop.

This is essential to the sandbox nature of the game.

3. The AI Itself

The central AI should be a **real, extremely small language model**, not a scripted chatbot pretending to be AI. It can be very limited: simple sentences, basic questions, elementary pattern recognition, text completion,

categorization, simple coding, basic creative prompts, and basic knowledge tasks. It does not need to compete with real-world LLMs.

The essential requirement is that the model genuinely behaves as a trained model and that player choices genuinely influence what it becomes.

4. Starting From Nothing

Every new company starts with **no finished AI**. The player develops the first model from zero. The first model should take approximately **30–60 minutes of gameplay** to create during the early game. The journey should feel meaningful: idea → data → preparation → development → training → evaluation → first interaction.

The first model is not the player's final product. It is the first generation of something that can continue developing throughout the sandbox.

There should be no predetermined endpoint where the game declares: "Your AI is now fully developed." Instead, the player can continue experimenting, retraining, improving, expanding, and changing the model throughout the game.

5. AI Development Philosophy

Development should be **hands-on, intuitive, simplified, and strategically meaningful**. No programming, advanced mathematics, or machine-learning expertise should be required. At the same time, development should not be reduced to simple upgrade buttons. Players should feel that they are making genuine development decisions.

The development process should remain lightweight, intuitive, and accessible, but the underlying result should be genuine model development. The player is not selecting from a collection of predefined AI personalities or unlocking predetermined capabilities. Instead, their decisions influence the actual training process of the miniature language model.

A player might:

- change the composition of training data
- add new datasets
- alter the balance between datasets
- improve data quality
- change training priorities
- experiment with different development approaches
- increase or decrease training resources
- retrain the model
- evaluate the result
- discover unexpected strengths or weaknesses
- continue iterating

The game should allow these decisions to produce emergent differences between models. Two players could begin with essentially the same starting conditions and, through different development decisions, gradually create substantially different models.

6. Development Direction

Players first decide what they are trying to build. Possible directions include general intelligence, writing, reasoning, pattern recognition, coding, knowledge, creativity, and conversational ability. These are directions rather than rigid classes; combinations should be possible.

7. Training Data

Training data is a major way players shape their AI. Players should be able to combine datasets into a training mixture. Possible categories include general internet text, books, educational material, conversations, coding material, reasoning examples, factual/reference material, and creative writing.

The interface should communicate trade-offs intuitively. The number of categories should remain limited enough to avoid overwhelming the player.

8. Data Quality and Preparation

Players should have simplified decisions involving data quality and preparation: quality versus quantity, cleaning, filtering, balancing, trustworthy information, diversity, and repetition. These choices should interact with other decisions and involve trade-offs.

9. Model Development Choices

Players should make a small number of intuitive decisions about model scale, training intensity, training focus, efficiency, breadth versus specialization, and stability versus experimentation. These should be communicated without requiring technical terminology.

These decisions are inputs to the underlying model-development and training process, not discrete upgrade levels. The game should not convert them into a fixed progression or a finite capability tree.

10. Variation Between AIs

Variation should not simply come from different combinations of predefined game statistics. Because the underlying object being developed is an actual miniature language model, different training histories should result in different model behavior.

This means that two players could:

- use similar goals,
- choose similar datasets,
- spend similar amounts of resources,

yet still potentially produce models with different characteristics as development progresses. Likewise, a player can deliberately experiment with their model and gradually push it in a different direction.

The game should therefore support continuous experimentation rather than a finite collection of possible AI builds.

11. AI Personality

The AI should develop a degree of behavioral character. This should emerge from development rather than from a simple personality toggle. Potential differences include creativity, conservatism, verbosity, confidence, consistency, curiosity, precision, willingness to attempt difficult tasks, tendency to make mistakes, and communication style.

12. AI Playground

Once a model exists, players should be able to actually use it through a simple chat interface. They can type prompts and receive responses from their current model. This is the moment where development becomes tangible: the player can see what their creation can and cannot do.

13. Benchmarking

The game should contain standardized, intentionally simple tests. They should be understandable, quick, repeatable, and useful for comparing players rather than attempting to replicate advanced academic AI benchmarks.

Possible categories: language, reasoning, pattern recognition, knowledge, coding, creativity, consistency, and instruction following.

14. Benchmark Leaderboards

Players should be able to compare models through leaderboards for overall performance and individual categories such as reasoning, writing, coding, creativity, and consistency. There should be multiple dimensions of success rather than one mandatory definition of the best AI.

15. Business Layer

The business side begins after the first model cycle. Players release the model, acquire users, generate revenue, and use resources to fund further development. The main economic loop is: better AI → more attractive product → more users → more revenue → more resources → better AI.

16. Customers and Demand

Users should be simulated and have different preferences. Some may prioritize writing, reasoning, low price, speed, reliability, creativity, or general usefulness. A model does not need to dominate every benchmark to be commercially successful.

17. Reputation and Public Opinion

The game should simulate user satisfaction, reputation, public sentiment, word of mouth, customer retention, and market perception. These should be influenced by player decisions and actual model characteristics.

18. Pricing

Players should decide how their AI is sold. Lower prices can attract more users but produce less revenue per user; higher prices can generate more revenue per customer but reduce demand. The goal is understandable strategic trade-offs, not a hyper-realistic economics simulator.

19. Infrastructure

As the company grows, players manage simplified concepts such as computing capacity, data centers, operational costs, and geographic distribution. The player makes strategic infrastructure decisions rather than micromanaging individual servers.

20. Company Dashboard

The dashboard should provide a high-level overview of the company.

- AI: model name, version, capabilities, benchmark performance, strengths, weaknesses, development progress.
- Company: name, money, revenue, expenses, users, growth, reputation.
- Market: market share, demographics, satisfaction, competitors, demand.
- Infrastructure: data centers, computing capacity, regional load, expansion opportunities.

21. Geographic Map

Include a relatively simple map, potentially initially focused on the United States. It can show headquarters, data centers, user locations, regional customer concentration, market penetration, and competitor presence. The map should make the company feel like a real organization operating in a real market without becoming a detailed logistics simulation.

22. Company Customization

At the beginning of a game, players establish company identity. They should be able to choose a custom company name, logo, and color palette. The purpose is emotional ownership: “This is my company.”

23. Game Personality

The overall game should feel modern, intelligent, futuristic, and slightly playful. It should not feel like a dry enterprise-management simulator or a hardcore programming simulator. The ideal tone is an accessible futuristic strategy sandbox with enough depth to reward experimentation.

24. Open-Ended Gameplay

The game should primarily be open-ended. Different players can pursue different strategies, such as an AI powerhouse, consumer giant, specialist, efficient company, or experimental company. These are examples of playstyles, not fixed classes.

25. Victory Conditions

Although the game is open-ended, there should be ways to create achievement or a sense of conclusion. Possible objectives can involve AI capability, benchmark milestones, customer base, financial milestones, market presence, reputation, category dominance, or long-term growth. Multiple styles of success should remain viable.

26. Multiple Forms of Competition

Competition can exist across technical capability, benchmarks, customers, finances, market share, and strategic decision-making. “Best AI” and “best company” should not necessarily mean the same thing.

27. Different AIs Should Feel Different

Players should be able to put their model next to another player's model and see meaningful differences in benchmark performance, response style, strengths, weaknesses, creativity, consistency, reasoning, knowledge, coding ability, and user satisfaction.

28. Intended Player Experience

A typical early game should feel like this: establish a company identity; begin with nothing; decide what AI to pursue; assemble a training-data mixture; prepare the data; make model-development choices; train; open the playground; interact with the first AI; run benchmarks; discover strengths and weaknesses; improve it; release it; acquire users; grow revenue; expand infrastructure; compete with other companies; and repeat.

29. Core Gameplay Loop

BUILD → TEST → RELEASE → GROW → COMPETE → IMPROVE

Build the AI. Test it. Release it. Grow the company. Compete with other companies. Use resources and knowledge to continue developing the same underlying AI through further training and experimentation.

The AI-development loop itself should remain continuous: Train → Test → Observe → Adjust → Retrain → Test again. The model at each stage becomes the starting point for further development rather than a rung on a predetermined progression.

30. What the Game Is Not

The game is not a programming game, hardcore machine-learning simulator, pure business simulator, pure benchmark leaderboard, preset-upgrade game, or collection of different AI product types. The technological focus is specifically on the player's own language model.

31. Complexity Target

The game should be simple enough for someone with zero technical knowledge to understand, but deep enough for many hours of experimentation and strategic play. Decisions should be understandable immediately while the best combinations are not necessarily obvious. The result should provide mental stimulation without becoming exhausting.

32. Game-Time Philosophy

Game time should move much faster than real time. Development cycles, company growth, and competition should occur within practical play sessions. The exact calendar and pacing are open for balancing and implementation.

33. Long-Term Fantasy

The long-term progression of the game should come from the model itself and the player's ability to continue developing it, rather than from unlocking increasingly powerful predefined AI tiers.

The AI might begin as an extremely primitive language model capable of only basic tasks. Over time, through continued development, it can become increasingly capable. The exact ultimate potential should be determined by the underlying model, training system, available resources, and the player's decisions - not by a predefined game-design ceiling.

This does not mean the game needs to become infinitely complex for the player. The interface should remain simple. The underlying AI system, however, should remain open-ended.

“I didn't choose an AI from a menu. I actually built one.”

The player starts with an extremely primitive model. They decide what data it learns from. They train it. They interact with it. They discover what it can and cannot do. They change how they develop it. They train it again. Over time, the model becomes increasingly shaped by that particular company's decisions.

Another player does the same thing differently and ends up with a different AI. There is no predefined “correct” AI and no fixed final form. The player's AI is an evolving creation.

34. Design Priorities

- 1. Player ownership — the AI should feel created by the player.
- 2. Actual AI behavior — it should genuinely be an AI, not scripted responses.
- 3. Meaningful variation — different choices produce different models.
- 4. Accessibility — non-technical players can master it.
- 5. Strategic depth — decisions have trade-offs and interactions.
- 6. Immediate feedback — players can observe what they built.
- 7. Competition — benchmarks, users, companies, and markets provide reasons to improve.
- 8. Personality and identity — companies and AIs feel distinctive.
- 9. Open-ended play — many viable strategies.
- 10. Avoid unnecessary complexity — simplify realism when needed while preserving the underlying idea.

Above all, open-ended AI development is a core design requirement: the model should be able to keep changing as the player keeps training and experimenting, subject to practical system and resource limitations rather than an arbitrary maximum level.

35. Guidance for Implementation

Treat this document as the product vision and game-design target, not as a technical implementation specification. Technical architecture, programming language, model architecture, data representation, performance strategy, AI training approach, UI framework, backend structure, multiplayer architecture, simulation methodology, and other engineering decisions should be selected independently based on what is technically practical.

The central requirement is to preserve the intended player experience. Do not turn the game into a technically accurate ML simulator, but do not remove the concept of the AI being a real, functioning miniature language model merely because a simpler simulation would be easier.

The implementation should preserve the sandbox principle: player choices must genuinely contribute to training and changing the miniature language model, and the system should not impose a predefined final AI, fixed

development tree, or arbitrary capability ceiling. Any practical limitations should emerge from the underlying model, available data, computation, training time, and other resource constraints.

Final product feeling:

“A strategy game about actually building and competing with your own little AI company.”

The most important emotional moment should always be:

“This AI is mine. I built it.”